Token Replacement in ASP.NET Core Web API

Back to ASP.NET Core Web API Series

Token Replacement in ASP.NET Core Web API

In this article, I will discuss **Token Replacement in ASP.NET Core Web API** with Examples. Please read our previous article, discussing how to Send Multiple JSON in a Single Response in the ASP.NET Core Web API before proceeding to this article.

In ASP.NET Core, **Attribute Routing** allows you to define routes directly above controller and action methods using attributes like [Route], [HttpGet], [HttpPost]. In these Routes, some fixed literals, such as "api/Employee", are hard-coded. **Token Replacement**, in ASP.NET Core, automatically substitutes parameters tokens into controller and action names.

What are Tokens?

Tokens are **predefined placeholders** enclosed in square brackets [] used within route templates. When a request matches a route template, the framework automatically replaces these tokens with actual values.

- [controller] – replaced by the controller name (minus the "Controller" suffix).
- [action] – replaced by the action method name.

Why Use Token Replacement?

Token replacement simplifies route management by:

- Reducing hard-coding and maintenance.
- Automatically updates routes if controller or action names change.
- Ensures consistency across multiple endpoints.
- Makes code easier to maintain, and less error-prone.

Real-time Analogy:

Imagine a **mail sorting system** where letters are addressed to "[name]@[street].com[city]". When you send letters to "Sam@Main.com", the system automatically substitutes those placeholders with the actual street's name, address, and city.

In the same way, ASP.NET Core uses tokens like [controller] and [action] to automatically fill in the actual controller and action names in the route.

Example Without Using Token Replacement in ASP.NET Core Web API:

Before understanding the need and use of token replacement, let's look at the concept of variable without it. Suppose we have two controllers, EmployeeController and DepartmentController, and we define routes for **GetAllEmployees** and **GetAll Departments** actions, as shown in the code below.

```
using Microsoft.AspNetCore.Mvc;
[ApiController]
[Route("Employee/GetAllEmployees")]
public class EmployeeController : ControllerBase
{
    [HttpGet]
    public IEnumerable<string> GetAll()
    {
        return "Response from GetAllEmployees Method";
    }
}

[ApiController]
[Route("Department/GetAllDepartments")]
public class DepartmentController : ControllerBase
{
    [HttpGet]
    public IEnumerable<string> GetAll()
    {
        return "Response from GetAllDepartments Method";
    }
}
```

Code Explanation:

- Each route is hard-coded.
- No explicit, specific Endpoints in Swagger (Swagger UI) will be created.
- If the controller name changes (e.g., to StaffController), all route strings must be updated manually.
- This approach increases the chance of errors and reduces maintainability.

Now, run the application and you can access both the resource using the controller and action method name, as shown in the image below.

Example Using Token Replacement in ASP.NET Core Web API

Let us understand how to use token replacement in the ASP.NET Core Web API with an example. Please modify the Employee Controller class as shown below. As you can see, we are using the token **[controller]** in the **[Route("Employee/[controller]")]** attribute.

```
using Microsoft.AspNetCore.Mvc;
[ApiController]
[Route("Employee/[controller]")]
public class EmployeeController : ControllerBase
{
    [HttpGet]
    public IEnumerable<string> GetAll()
    {
        return "Response from GetAllEmployees Method";
    }
}
```

Now, run the above changes in Swagger, you can access the **GetAllEmployees** method with the URL **/Employee/GetAllEmployees** and the **GetAllDepartments** method with the URL **/Employee/GetAllDepartments**, as shown in the image below.

What Happens at Runtime?

Now, at runtime:

- [controller] – replaced by Employee
- [action] – replaced by the method name (GetAllEmployees, GetAllDepartments).

So, the URL becomes:

- /Employee/GetAllEmployees
- /Employee/GetAllDepartments

Now, with the above changes in place, you can access the **GetAllEmployees** method with the URL **/Employee/GetAllEmployees** and the **GetAllDepartments** method with the URL **/Employee/GetAllDepartments**, as shown in the image below.

Routing Advantages:

- Prior to this using **extensive file naming rules**, if your system e.g. your software automatically name each request as "Employee.Get.Request.cs".
- If your Application changes the endpoint, **Swagger** will be automatically updated.
- Controller, [controller] and [action] dynamically adjust route names, no need to hardcode or manually update them.

Do we need to write the action name in each action method?

Not necessarily. If you want the [action] token to apply to **all action methods** automatically, you can declare it once at the controller level using the **[Route("Controller/[action]")]** attribute.

Let us understand this with an example. Please modify the Employee Controller class as shown in the code below. As you can see, we are using the token **[action]** in the **[Route("Employee/[action]")]** attribute at the controller level and omitted the Route attribute on **[Route("GetAllEmployees")]** which is present on the action level.

```
using Microsoft.AspNetCore.Mvc;
[ApiController]
[Route("Employee/[action]")]
public class EmployeeController : ControllerBase
{
    [HttpGet]
    public IEnumerable<string> GetAll()
    {
        return "Response from GetAllEmployees Method";
    }
}
```

What Happens Now:

- Prior to this using **extensive file naming rules**, if your system e.g. your software automatically name each request as "Employee.Get.Request.cs".
- If your Application changes the endpoint, **Swagger** will be automatically updated.
- Controller, [controller] and [action] dynamically adjust route names, no need to hardcode or manually update them.

Do we need to write the action name in each action method?

Not necessarily. If you want the [action] token to apply to **all action methods** automatically, you can declare it once at the controller level using the **[Route("Controller/[action]")]** attribute.

Let us understand this with an example. Please modify the Employee Controller class as shown in the code below. As you can see, we are using the token **[action]** in the **[Route("Employee/[action]")]** attribute at the controller level and omitted the Route attribute on **[Route("GetAllEmployees")]** which is present on the action level.

```
using Microsoft.AspNetCore.Mvc;
[ApiController]
[Route("Employee/[action]")]
public class EmployeeController : ControllerBase
{
    [HttpGet]
    public IEnumerable<string> GetAll()
    {
        return "Response from GetAllEmployees Method";
    }
}
```

What Happens Now:

- The controller level route simplifies [controller]/[action] syntax in all action methods.
- Action-level overrides remain possible if needed (e.g., custom route for one method).
- Ensures consistent and predictable URL structure across all endpoints.

Routing Advantages:

- Prior to this using **extensive file naming rules**, if your system e.g. your software automatically name each request as "Employee.Get.Request.cs".
- If your Application changes the endpoint, **Swagger** will be automatically updated.
- Controller, [controller] and [action] dynamically adjust route names, no need to hardcode or manually update them.

Controller and Action level route applied to the action method:

You can also apply [controller]/[action] directly on each method instead of at the controller level. In this example, here, we apply the Route Attribute on the action method level, and it works as expected.

```
using Microsoft.AspNetCore.Mvc;
[ApiController]
[Route("Employee/[action]")]
public class EmployeeController : ControllerBase
{
    [HttpGet]
    public IEnumerable<string> GetAll()
    {
        return "Response from GetAllEmployees Method";
    }
    [HttpGet]
    public IEnumerable<string> GetAllDepartments()
    {
        return "Response from GetAllDepartments Method";
    }
}
```

Code Explanation:

- Static route [controller] and [action] tokens are used at the **method level**.
- Each method explicitly defines its route using the controller and action tokens with **route parameters**.
- The framework will automatically substitute the actual values at runtime:
  - /Employee/GetAllEmployees
  - /Employee/GetAllDepartments
  - /Employee/GetAllDepartments

When to Use:

- This approach is useful when **single methods follow a different URL pattern**.
- You have complex controller-level route conflicts.

Routing Advantages:

- Prior to this using **extensive file naming rules**, if your system e.g. your software automatically name each request as "Employee.Get.Request.cs".
- If your Application changes the endpoint, **Swagger** will be automatically updated.
- Controller, [controller] and [action] dynamically adjust route names, no need to hardcode or manually update them.

Disadvantages:

- Requires manual updates if controller or action names change.
- Increases maintenance effort compared to centralized routing.
- Can lead to inconsistent URL patterns if not carefully managed.

In the next article, I will discuss **Route Prefix in ASP.NET Core Web API** with Examples. In this article, I try to explain token replacement in ASP.NET Core Web API with Examples. I hope you will enjoy with this article. I hope this article will help you to understand Token Replacement in ASP.NET Core Web API with Examples. Please provide your valuable feedback.

Previous Lesson: Multiple JSON in a Single Response in ASP.NET Core Web API

Next Lesson: Route Prefix in ASP.NET Core Web API

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

Post Comment

© Dot Net Tutorials | Website Designed by Suresh Pasal